Regresión en TensorFlow

Guía académica 01 · Del gradiente escrito a mano a `tf.keras`

Juan Sebastián Laverde Chunza

15 de octubre de 2026

Esta guía estudia la regresión con redes neuronales en TensorFlow en dos etapas. Primero se programa el entrenamiento a mano: parámetros, predicción, pérdida, gradientes y actualización. Después se traslada el mismo problema a `tf.keras`, donde se revisan tres formas de definir modelos, capas propias, activaciones, *callbacks*, guardado de modelos, regularización y normalización. Todo el código de la guía se ejecutó al generar el documento, y las salidas que aparecen son reales.

Tabla de contenidos

1 Propósito y mapa de lectura **1**

2 Intuición: aprender es corregir con método **2**

3 Fundamentos matemáticos **2**
3.1 Modelo, pérdida y riesgo empírico . . . 2
3.2 Gradiente del modelo lineal . . . 2
3.3 Ejemplo numérico resuelto . . . 3
3.4 Más allá de la línea recta . . . 4

4 Datos: generación, partición y `tf.data` **4**

5 Entrenamiento manual con `tf.GradientTape` **5**
5.1 Paso 1: un modelo lineal como punto de partida . . . 5
5.2 Paso 2: una red de una capa oculta, escrita a mano . . . 6

6 `tf.keras`: tres formas de definir el mismo modelo **8**

7 Capas personalizadas **9**

8 Funciones de activación **9**

9 `compile()`, `fit()` y el objeto `History` **10**

10 *Callbacks*: intervenir durante el entrenamiento **11**

11 Evaluación y predicción **13**

12 Guardar y cargar modelos **13**

13 Regularización **14**
13.1 Penalizaciones L1, L2 y L1-L2 . . . 14

13.2 *Dropout* 15

14 Normalización de activaciones **16**

15 Dimensiones de tensores y flujo de datos **17**

16 Hiperparámetros del modelo **19**

17 Errores frecuentes **20**

18 Ejercicios **20**
18.1 Respuestas orientativas 21

19 Glosario **21**

20 Conclusiones académicas **22**

21 Referencias **23**

Procedencia y autoría

Autor de esta guía. Juan Sebastián Laverde Chunza, con apoyo de herramientas de inteligencia artificial generativa. Es material académico complementario y **no** es material oficial del docente ni de la Universidad El Bosque.
Referencia académica. Los temas siguen el notebook `01_Regresion.ipynb` de la Clase 02 del curso Deep Learning I, del profesor **Héctor Javier Hortúa, PhD** (GitHub · Colab).
Edición pública. El código, los datos, los ejemplos y las explicaciones de esta edición son propios e independientes: no reproducen el código ni el texto del notebook del docente. Citar el notebook como referencia no equivale a una licencia ni a una autorización sobre el material original.

1. Propósito y mapa de lectura

Un problema de **regresión** busca predecir un número: un precio, una temperatura o una dosis. Una red neuronal resuelve ese problema ajustando parámetros para que sus predicciones se parezcan a los valores observados. Esta guía separa el proceso en piezas y las construye una a una.

Bloque	Pregunta que responde	Herramienta
Fundamentos (§3)	¿Qué se minimiza y cómo se calcula el gradiente?	Álgebra y cálculo
Datos (§4)	¿Cómo se separan y se sirven los datos?	NumPy, `tf.data`
Entrenamiento manual (§5)	¿Qué hace un *optimizer* por dentro?	`tf.Variable`, `tf.GradientTape`
`tf.keras` (§6–§12)	¿Cómo se expresa lo mismo con la API de alto nivel?	`Sequential`, API funcional, *subclassing*
Generalización (§13–§14)	¿Cómo se evita el sobreajuste y se estabiliza el entrenamiento?	L1/L2, *Dropout*, normalizaciones

Si no programas, lee las secciones 2 y 3 y los recuadros de interpretación. **Si programas**, ejecuta el código en el orden en que aparece: cada bloque depende de los anteriores.

2. Intuición: aprender es corregir con método

Imagina que ajustas la perilla de una radio para sintonizar una emisora. Giras un poco, escuchas si mejora o empeora y vuelves a girar en la dirección que mejora. Entrenar un modelo es lo mismo, con miles de perillas a la vez:

1. **Predecir.** El modelo produce una estimación con sus parámetros actuales.
2. **Medir.** Una *función de pérdida* resume en un solo número qué tan lejos está la estimación de la respuesta real.
3. **Diagnosticar.** El *gradiente* indica, para cada parámetro, hacia dónde y con qué intensidad cambia la pérdida.
4. **Corregir.** Cada parámetro se mueve un paso pequeño en la dirección que reduce la pérdida.

Ese ciclo se repite hasta que la pérdida deja de bajar. La parte delicada es comprobar que el modelo también acierta con datos que **no** vio al ajustarse. Para eso existen los conjuntos de validación y de prueba.

3. Fundamentos matemáticos

3.1. Modelo, pérdida y riesgo empírico

Se dispone de n pares (x_i, y_i). Un modelo con parámetros θ produce $\hat{y}_i = f_\theta(x_i)$. En regresión, la pérdida más usada es el **error cuadrático medio**:

$$\mathcal{L}(\theta) \;=\; \frac{1}{n}\sum_{i=1}^{n}\left(\hat{y}_i - y_i\right)^2.$$

Elevar al cuadrado penaliza más los errores grandes y produce una función suave, fácil de derivar. Una alternativa robusta frente a valores atípicos es el **error absoluto medio**, $\frac{1}{n}\sum |\hat{y}_i - y_i|$.

3.2. Gradiente del modelo lineal

Para el modelo lineal $\hat{y}_i = w\,x_i + b$, la regla de la cadena da

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{n}\sum_{i=1}^{n}\left(\hat{y}_i - y_i\right)x_i, \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}\left(\hat{y}_i - y_i\right).$$

El **descenso de gradiente** actualiza cada parámetro con una tasa de aprendizaje $\eta > 0$:

$$w \leftarrow w - \eta\,\frac{\partial \mathcal{L}}{\partial w}, \qquad b \leftarrow b - \eta\,\frac{\partial \mathcal{L}}{\partial b}.$$

3.3. Ejemplo numérico resuelto

Considera tres puntos (x, y): $(1,\,3)$, $(2,\,5)$ y $(3,\,6)$, y los parámetros iniciales $w = 1$, $b = 0$, con $\eta = 0.1$.

x	y	$\hat{y} = wx + b$	$\hat{y} - y$	$(\hat{y} - y)\,x$
1	3	1	-2	-2

x	y	$\hat{y} = wx + b$	$\hat{y} - y$	$(\hat{y} - y)\, x$
2	5	2	-3	-6
3	6	3	-3	-9

- Pérdida: $\mathcal{L} = (4 + 9 + 9)/3 = 7.333$.
- Gradientes: $\partial_w \mathcal{L} = \frac{2}{3}(-17) = -11.333$ y $\partial_b \mathcal{L} = \frac{2}{3}(-8) = -5.333$.
- Actualización: $w = 1 + 0.1 \cdot 11.333 = 2.133$ y $b = 0 + 0.1 \cdot 5.333 = 0.533$.

Con los nuevos parámetros, las predicciones son 2.67, 4.80 y 6.93, y la pérdida baja a 0.34. Un solo paso redujo la pérdida más de veinte veces. El código siguiente comprueba estas cuentas con diferenciación automática.

Desde TensorFlow 2.16, `tf.keras` corresponde a **Keras 3**. Por eso se importa directamente con `import keras`; el resto del código usa TensorFlow para tensores, gradientes y `tf.data`.

```
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import keras  # en TensorFlow >= 2.16, tf.keras es Keras 3

tf.get_logger().setLevel("ERROR")   # oculta avisos de retrazado
keras.utils.set_random_seed(2026)
print("TensorFlow", tf.__version__, "| Keras", keras.__version__)
```

```
TensorFlow 2.21.0 | Keras 3.15.1
```

```
xs = tf.constant([1.0, 2.0, 3.0])
ys = tf.constant([3.0, 5.0, 6.0])
w = tf.Variable(1.0)
b = tf.Variable(0.0)

with tf.GradientTape() as tape:
    loss = tf.reduce_mean(tf.square(w * xs + b - ys))
grad_w, grad_b = tape.gradient(loss, [w, b])
print(f"perdida={loss.numpy():.3f}  dL/dw={grad_w.numpy():.3f}  "
      f"dL/db={grad_b.numpy():.3f}")

w.assign_sub(0.1 * grad_w)
b.assign_sub(0.1 * grad_b)
new_loss = tf.reduce_mean(tf.square(w * xs + b - ys))
print(f"w={w.numpy():.3f}  b={b.numpy():.3f}  "
      f"nueva perdida={new_loss.numpy():.3f}")
```

```
perdida=7.333  dL/dw=-11.333  dL/db=-5.333
w=2.133  b=0.533  nueva perdida=0.341
```

3.4. Más allá de la línea recta

Una red con una capa oculta de h neuronas calcula

$$\hat{y} = \mathbf{w}_2^\top \, \sigma(\mathbf{W}_1 x + \mathbf{b}_1) + b_2,$$

donde σ es una función de activación no lineal (ReLU, tanh, etc.). Sin σ, la composición de capas lineales sigue siendo lineal. La activación es lo que permite aproximar curvas. En redes con miles de parámetros no se derivan las fórmulas a mano: TensorFlow registra las operaciones de la pasada hacia delante y aplica la regla de la cadena en sentido inverso. Ese procedimiento se llama **diferenciación automática en modo reverso**.

4. Datos: generación, partición y `tf.data`

Para que la guía funcione en cualquier equipo, se genera un conjunto sintético con una tendencia lineal y una oscilación. La función verdadera es $y = 0.6\,x + 1.4\,\sin(1.8\,x) + \varepsilon$ con $\varepsilon \sim \mathcal{N}(0,\, 0.25^2)$.

```
rng = np.random.default_rng(11)

def make_wave_dataset(n_points, rng):
    x = rng.uniform(-3.0, 3.0, size=(n_points, 1)).astype("float32")
    noise = rng.normal(0.0, 0.25, size=(n_points, 1)).astype("float32")
    y = 0.6 * x + 1.4 * np.sin(1.8 * x) + noise
    return x, y.astype("float32")

x_all, y_all = make_wave_dataset(1200, rng)

order = rng.permutation(len(x_all))
n_fit, n_check = int(0.70 * len(order)), int(0.15 * len(order))
idx_fit = order[:n_fit]
idx_check = order[n_fit:n_fit + n_check]
idx_final = order[n_fit + n_check:]

x_fit, y_fit = x_all[idx_fit], y_all[idx_fit]
x_check, y_check = x_all[idx_check], y_all[idx_check]
x_final, y_final = x_all[idx_final], y_all[idx_final]
print("entrenamiento:", x_fit.shape, "| validacion:", x_check.shape,
      "| prueba:", x_final.shape)
```

```
entrenamiento: (840, 1) | validacion: (180, 1) | prueba: (180, 1)
```

Los tres subconjuntos cumplen papeles distintos:

- **Entrenamiento:** los parámetros se ajustan con estos datos.
- **Validación:** sirve para comparar configuraciones y decidir cuándo detenerse.
- **Prueba:** se usa una sola vez, al final, para estimar el desempeño con datos nuevos.

Si se consulta la prueba para tomar decisiones, deja de ser una estimación honesta.

```
grid = np.linspace(-3, 3, 300, dtype="float32").reshape(-1, 1)
truth = 0.6 * grid + 1.4 * np.sin(1.8 * grid)
fig, ax = plt.subplots()
ax.scatter(x_fit, y_fit, s=6, alpha=0.4, label="entrenamiento")
ax.scatter(x_check, y_check, s=6, alpha=0.6, label="validación")
ax.plot(grid, truth, color="black", lw=1.5, label="función verdadera")
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.legend(loc="upper left")
plt.tight_layout(); plt.show()
```

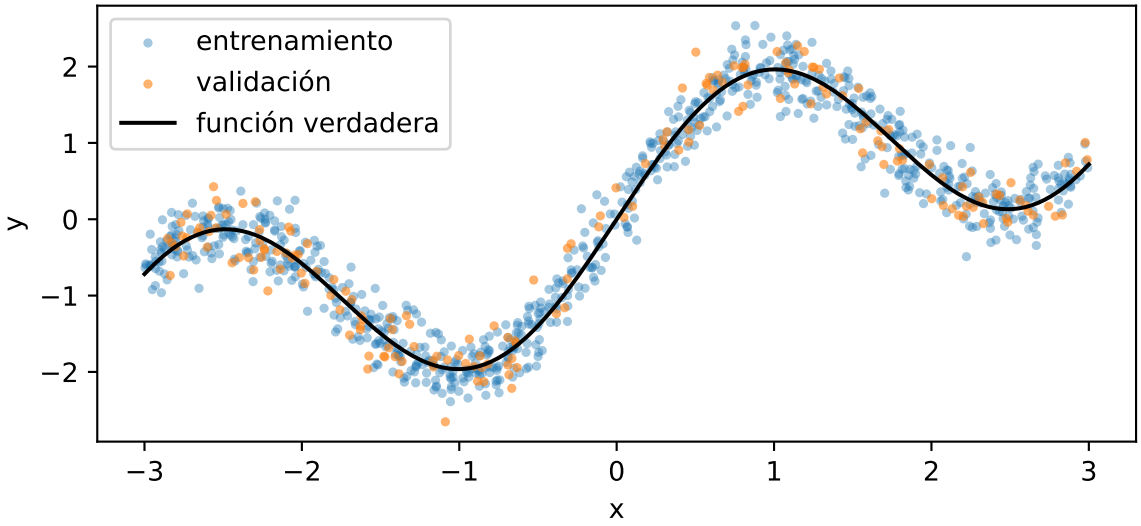


Figura 1: Datos sintéticos de entrenamiento y validación, con la función verdadera sin ruido.

`tf.data` convierte los arreglos en un flujo de **lotes** (*batches*). `shuffle` mezcla el orden en cada época para que el modelo no memorice la secuencia, `batch` agrupa ejemplos y `prefetch` prepara el siguiente lote mientras se procesa el actual.

```
BATCH = 64

def to_stream(x, y, training):
    stream = tf.data.Dataset.from_tensor_slices((x, y))
    if training:
        stream = stream.shuffle(buffer_size=len(x), seed=5,
                                reshuffle_each_iteration=True)
    return stream.batch(BATCH).prefetch(tf.data.AUTOTUNE)

fit_stream = to_stream(x_fit, y_fit, training=True)
check_stream = to_stream(x_check, y_check, training=False)
final_stream = to_stream(x_final, y_final, training=False)

first_x, first_y = next(iter(fit_stream))
print("lote de entrada:", first_x.shape, "| lote de salida:", first_y.shape)
```

```
lote de entrada: (64, 1) | lote de salida: (64, 1)
```

5. Entrenamiento manual con `tf.GradientTape`

5.1. Paso 1: un modelo lineal como punto de partida

Antes de usar una red, conviene medir qué logra el modelo más simple. Los parámetros son `tf.Variable` porque deben cambiar durante el entrenamiento; un `tf.constant` no admite asignaciones.

```
slope = tf.Variable(0.0)
intercept = tf.Variable(0.0)
```

```
def mse(pred, target):
    return tf.reduce_mean(tf.square(pred - target))

def linear_step(xb, yb, lr=0.05):
    with tf.GradientTape() as tape:
        loss = mse(slope * xb + intercept, yb)
    d_slope, d_icpt = tape.gradient(loss, [slope, intercept])
    slope.assign_sub(lr * d_slope)
    intercept.assign_sub(lr * d_icpt)
    return loss

for epoch in range(30):
    for xb, yb in fit_stream:
        linear_step(xb, yb)

linear_val = mse(slope * x_check + intercept, y_check).numpy()
print(f"pendiente={slope.numpy():.3f}  intercepto={intercept.numpy():.3f}  "
      f"MSE validacion={linear_val:.3f}")
```

```
pendiente=0.420  intercepto=-0.039  MSE validacion=1.099
```

La pendiente estimada no coincide con la tendencia verdadera de 0.6. La recta intenta absorber también parte de la oscilación, y la mejor recta posible no es la tendencia de la función. Además, el error de validación queda muy por encima de la varianza del ruido ($0.25^2 = 0.0625$). Eso es **subajuste**: el modelo es demasiado simple para el patrón.

5.2. Paso 2: una red de una capa oculta, escrita a mano

Se definen las matrices de pesos con inicialización de Glorot. Esta inicialización mantiene la varianza de las activaciones en un rango razonable. El bucle de entrenamiento hace explícito lo que `fit()` hace por dentro.

```
HIDDEN = 32
init = keras.initializers.GlorotUniform(seed=3)
W_in = tf.Variable(init(shape=(1, HIDDEN)), name="W_in")
b_in = tf.Variable(tf.zeros(HIDDEN), name="b_in")
W_out = tf.Variable(init(shape=(HIDDEN, 1)), name="W_out")
b_out = tf.Variable(tf.zeros(1), name="b_out")
params = [W_in, b_in, W_out, b_out]

def tiny_net(x):
    hidden = tf.tanh(x @ W_in + b_in)
    return hidden @ W_out + b_out

opt = keras.optimizers.Adam(learning_rate=0.01)

@tf.function
def train_step(xb, yb):
    with tf.GradientTape() as tape:
        loss = mse(tiny_net(xb), yb)
    grads = tape.gradient(loss, params)
    opt.apply_gradients(zip(grads, params))
    return loss

curve_fit, curve_check = [], []
```

```
for epoch in range(150):
    batch_losses = [train_step(xb, yb) for xb, yb in fit_stream]
    curve_fit.append(float(tf.reduce_mean(batch_losses)))
    curve_check.append(float(mse(tiny_net(x_check), y_check)))

print(f"MSE entrenamiento={curve_fit[-1]:.4f} | "
      f"MSE validacion={curve_check[-1]:.4f}")
```

```
MSE entrenamiento=0.0940 | MSE validacion=0.1038
```

El decorador `@tf.function` convierte la función de Python en un **grafo** de TensorFlow. La primera llamada lo traza y las siguientes lo reutilizan, lo que acelera el bucle. `opt.apply_gradients` hace la actualización de Adam: combina el gradiente con promedios móviles de su primer y segundo momento.

```
fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.9))
axes[0].plot(curve_fit, label="entrenamiento")
axes[0].plot(curve_check, label="validación")
axes[0].set_yscale("log"); axes[0].set_xlabel("época")
axes[0].set_ylabel("MSE"); axes[0].legend()
axes[1].scatter(x_check, y_check, s=6, alpha=0.4, color="gray")
axes[1].plot(grid, tiny_net(grid).numpy(), label="red (32 tanh)")
axes[1].plot(grid, slope.numpy() * grid + intercept.numpy(), "--",
             label="recta")
axes[1].set_xlabel("x"); axes[1].legend()
plt.tight_layout(); plt.show()
```

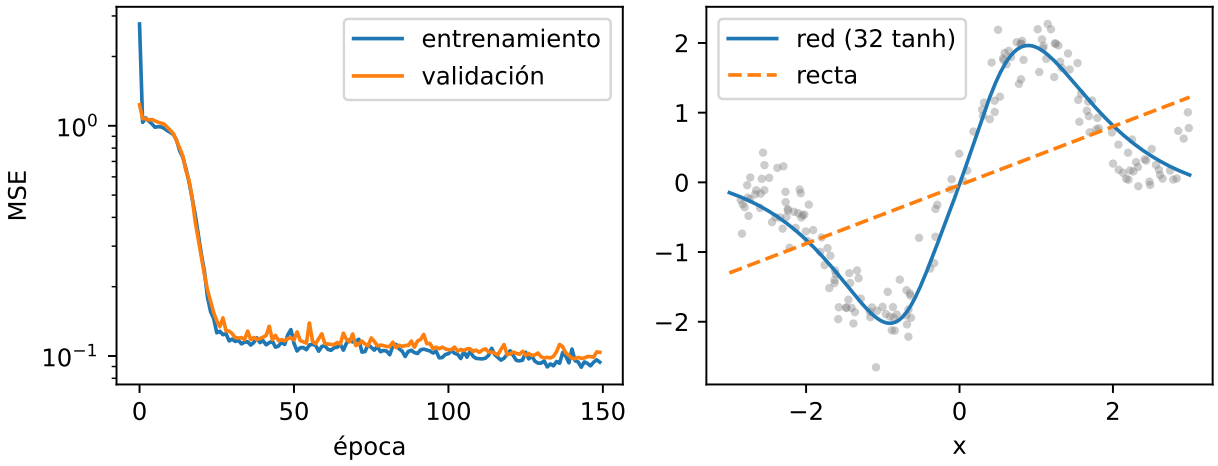


Figura 2: Izquierda: pérdida por época del entrenamiento manual. Derecha: ajuste de la red frente a la recta.

Cómo leer el resultado

La validación sigue de cerca al entrenamiento: la red no está memorizando. Ambas pérdidas quedan por encima de 0.0625, la varianza del ruido. Ningún modelo puede bajar de ese valor de forma sistemática, porque el ruido es impredecible por construcción. La distancia restante se puede reducir con más neuronas o más épocas. Si el error de entrenamiento cayera por debajo de 0.0625 mientras el de validación sube, habría **sobreajuste**.

6. `tf.keras`: tres formas de definir el mismo modelo

Keras encapsula capas, pérdidas, optimizadores y el bucle de entrenamiento. Un mismo modelo puede escribirse de tres maneras, y conviene elegir según la complejidad de la arquitectura.

Estilo	Cuándo usarlo	Limitación
`Sequential`	Pila lineal de capas	Una sola entrada y una sola salida, sin ramas
API funcional	Grafos con ramas, varias entradas o salidas, conexiones residuales	El grafo debe ser estático
Subclassing	Lógica arbitraria en `call()`, como bucles o condiciones	Menos introspección; hay que definir `get_config` para guardar

```python
def stack_model():
    return keras.Sequential([
        keras.Input(shape=(1,)),
        keras.layers.Dense(48, activation="relu"),
        keras.layers.Dense(48, activation="relu"),
        keras.layers.Dense(1),
    ], name="pila")

def graph_model():
    inputs = keras.Input(shape=(1,), name="x")
    h = keras.layers.Dense(48, activation="relu")(inputs)
    h = keras.layers.Dense(48, activation="relu")(h)
    outputs = keras.layers.Dense(1, name="y_hat")(h)
    return keras.Model(inputs, outputs, name="grafo")

class TwoBlockRegressor(keras.Model):
    def __init__(self, width=48, **kwargs):
        super().__init__(**kwargs)
        self.block_a = keras.layers.Dense(width, activation="relu")
        self.block_b = keras.layers.Dense(width, activation="relu")
        self.head = keras.layers.Dense(1)

    def call(self, x):
        return self.head(self.block_b(self.block_a(x)))

variants = {"Sequential": stack_model(), "Funcional": graph_model(),
            "Subclase": TwoBlockRegressor(name="subclase")}
for label, model in variants.items():
    model(tf.zeros((1, 1)))  # construye los pesos
    print(f"{label:<11} parametros entrenables: {model.count_params()}")
```

```
Sequential  parametros entrenables: 2497
Funcional   parametros entrenables: 2497
Subclase    parametros entrenables: 2497
```

Los tres modelos tienen exactamente los mismos parámetros: $(1 \cdot 48 + 48) + (48 \cdot 48 + 48) + (48 \cdot 1 + 1) = 2\,497$. La diferencia está en cómo se declaran.

7. Capas personalizadas

Una capa propia hereda de `keras.layers.Layer`. Crea sus pesos en `build()`, cuando ya se conoce la forma de la entrada, y define el cálculo en `call()`. Como ejemplo se construye una capa de **características senoidales entrenables**: transforma x en $\sin(\omega_k x + \varphi_k)$, con frecuencias ω_k y fases φ_k aprendidas. Es útil cuando el fenómeno tiene periodicidades, como en nuestros datos.

```
@keras.saving.register_keras_serializable(package="guia01")
class SineFeatures(keras.layers.Layer):
    """Proyecta x en sin(omega * x + phase), con omega y phase entrenables."""

    def __init__(self, n_waves=16, **kwargs):
        super().__init__(**kwargs)
        self.n_waves = n_waves

    def build(self, input_shape):
        self.omega = self.add_weight(
            name="omega", shape=(input_shape[-1], self.n_waves),
            initializer=keras.initializers.RandomNormal(stddev=1.5, seed=9))
        self.phase = self.add_weight(
            name="phase", shape=(self.n_waves,), initializer="zeros")

    def call(self, inputs):
        return tf.sin(inputs @ self.omega + self.phase)

    def get_config(self):
        return {**super().get_config(), "n_waves": self.n_waves}

inputs = keras.Input(shape=(1,))
waves = SineFeatures(16, name="ondas")(inputs)
mixed = keras.layers.Concatenate()([inputs, waves])
outputs = keras.layers.Dense(1)(mixed)
wave_model = keras.Model(inputs, outputs, name="modelo_ondas")
print("parametros:", wave_model.count_params())
```

```
parametros: 50
```

El modelo combina la entrada original, que aporta la tendencia lineal, con 16 ondas aprendidas, que aportan la oscilación. Tiene apenas 50 parámetros, frente a los 2 497 de la red densa.

8. Funciones de activación

La activación decide qué forma pueden tomar las funciones que la red representa.

Activación	Fórmula	Rango	Comentario
ReLU	$\max(0, z)$	$[0, \infty)$	Barata; puede dejar neuronas «muertas» si $z < 0$ siempre
tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Centrada en cero; se satura para $\lvert z \rvert$ grande

Activación	Fórmula	Rango	Comentario
Sigmoide	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Útil en salidas de probabilidad, rara en capas ocultas
GELU	$z\,\Phi(z)$	$\approx (-0.17, \infty)$	Suave; habitual en *transformers*
Swish/SiLU	$z\,\sigma(z)$	$\approx (-0.28, \infty)$	Suave y no monótona

En regresión, la **capa de salida** suele ser lineal (sin activación), para que la red pueda producir cualquier valor real. El experimento siguiente entrena la misma arquitectura con cuatro activaciones durante el mismo número de épocas.

```
def dense_regressor(act, width=48, name=None):
    return keras.Sequential([
        keras.Input(shape=(1,)),
        keras.layers.Dense(width, activation=act),
        keras.layers.Dense(width, activation=act),
        keras.layers.Dense(1),
    ], name=name)

results = {}
for act in ["relu", "tanh", "gelu", "silu"]:
    keras.utils.set_random_seed(21)
    net = dense_regressor(act)
    net.compile(optimizer=keras.optimizers.Adam(0.005), loss="mse")
    net.fit(fit_stream, epochs=60, verbose=0)
    results[act] = net.evaluate(check_stream, verbose=0)

for act, val in sorted(results.items(), key=lambda kv: kv[1]):
    print(f"{act:<5} MSE validacion = {val:.4f}")
```

```
gelu  MSE validacion = 0.0743
silu  MSE validacion = 0.0777
relu  MSE validacion = 0.0800
tanh  MSE validacion = 0.1176
```

En esta ejecución, GELU y SiLU obtienen el menor error, ReLU queda cerca y tanh es la más lenta con el mismo presupuesto de 60 épocas. Una explicación plausible es que tanh se satura para entradas grandes y sus gradientes se hacen pequeños. Estas diferencias dependen del problema, de la inicialización y del número de épocas. El experimento enseña un **método de comparación** (misma arquitectura, misma semilla, mismo presupuesto), no una regla general.

9. `compile()`, `fit()` y el objeto `History`

`compile()` fija tres piezas: el **optimizador**, la **pérdida** y las **métricas** que se reportan. `fit()` ejecuta el bucle por épocas y devuelve un objeto `History`, cuyo diccionario `history` guarda las pérdidas y métricas de cada época.

```
keras.utils.set_random_seed(4)
main_model = dense_regressor("tanh", name="regresor_principal")
main_model.compile(
```

```
    optimizer=keras.optimizers.Adam(learning_rate=0.01),
    loss=keras.losses.MeanSquaredError(),
    metrics=[keras.metrics.MeanAbsoluteError(name="mae"),
             keras.metrics.RootMeanSquaredError(name="rmse")],
)
history = main_model.fit(fit_stream, validation_data=check_stream,
                         epochs=40, verbose=0)
print("claves de History:", sorted(history.history))
for key, values in history.history.items():
    print(f"  {key:<9} ultima epoca = {values[-1]:.4f}")
```

```
claves de History: ['loss', 'mae', 'rmse', 'val_loss', 'val_mae', 'val_rmse']
  loss      ultima epoca = 0.0942
  mae       ultima epoca = 0.2490
  rmse      ultima epoca = 0.3070
  val_loss  ultima epoca = 0.0984
  val_mae   ultima epoca = 0.2452
  val_rmse  ultima epoca = 0.3136
```

10. *Callbacks*: intervenir durante el entrenamiento

Un *callback* es un objeto que Keras llama en momentos definidos: al iniciar o terminar una época, un lote o el entrenamiento completo. Se usan los siguientes:

- `EarlyStopping` detiene el entrenamiento si la métrica vigilada no mejora durante `patience` épocas y, con `restore_best_weights=True`, recupera los mejores pesos.
- `ReduceLROnPlateau` divide la tasa de aprendizaje cuando la validación se estanca.
- `ModelCheckpoint` guarda el mejor modelo en disco.
- `TensorBoard` escribe registros que luego se exploran con `tensorboard --logdir`.
- Un *callback* propio con `LambdaCallback` imprime un resumen cada cierto número de épocas.

```
import tempfile, pathlib
workdir = pathlib.Path(tempfile.mkdtemp(prefix="guia01_"))

def every_25(epoch, logs):
    if (epoch + 1) % 25 == 0:
        lr = float(keras.ops.convert_to_numpy(
            main_model.optimizer.learning_rate))
        print(f"  epoca {epoch + 1:>3}: val_loss={logs['val_loss']:.4f}"
              f"  lr={lr:.5f}")

report = keras.callbacks.LambdaCallback(on_epoch_end=every_25)

watchers = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=20,
                                  restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
                                      patience=8, min_lr=1e-4),
    keras.callbacks.ModelCheckpoint(workdir / "mejor.keras",
                                    monitor="val_loss", save_best_only=True),
    keras.callbacks.TensorBoard(log_dir=str(workdir / "logs")),
    report,
]
```

```
history_cb = main_model.fit(fit_stream, validation_data=check_stream,
                            epochs=300, callbacks=watchers, verbose=0)
print("epocas ejecutadas:", len(history_cb.history["loss"]))
print("archivos en la carpeta de trabajo:",
      sorted(p.name for p in workdir.iterdir()))
```

```
  epoca  25: val_loss=0.0927  lr=0.01000
  epoca  50: val_loss=0.0885  lr=0.00250
  epoca  75: val_loss=0.0855  lr=0.00125
  epoca 100: val_loss=0.0846  lr=0.00031
epocas ejecutadas: 105
archivos en la carpeta de trabajo: ['logs', 'mejor.keras']
```

```
hist = history_cb.history
best_epoch = int(np.argmin(hist["val_loss"]))
fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.8))
axes[0].plot(hist["loss"], label="entrenamiento")
axes[0].plot(hist["val_loss"], label="validación")
axes[0].axvline(best_epoch, ls=":", color="black")
axes[0].set_yscale("log"); axes[0].set_xlabel("época")
axes[0].set_ylabel("MSE"); axes[0].legend()
axes[1].plot(hist["learning_rate"])
axes[1].set_yscale("log"); axes[1].set_xlabel("época")
axes[1].set_ylabel("tasa de aprendizaje")
plt.tight_layout(); plt.show()
```

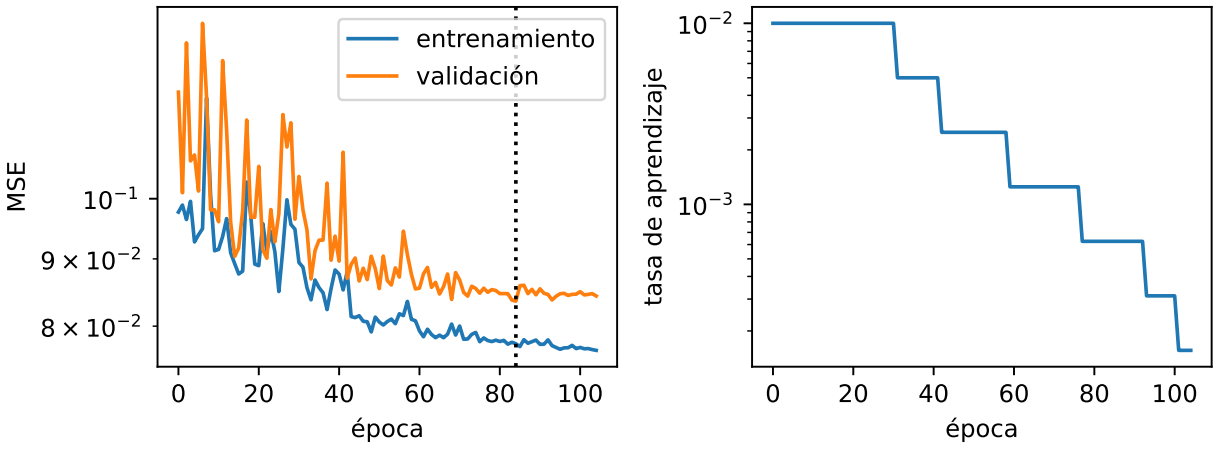


Figura 3: Curvas de entrenamiento con callbacks. La línea punteada marca la época con menor pérdida de validación.

La curva derecha muestra los escalones de `ReduceLROnPlateau`: cada vez que la validación deja de mejorar, la tasa se reduce a la mitad y el optimizador da pasos más finos.

11. Evaluación y predicción

`evaluate()` calcula la pérdida y las métricas sobre un conjunto completo; `predict()` devuelve las salidas del modelo. Además del MSE, se reporta el **coeficiente de determinación** $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$, que compara el modelo con la estrategia ingenua de predecir siempre la media.

```
wave_model.compile(optimizer=keras.optimizers.Adam(0.02), loss="mse")
wave_model.fit(fit_stream, validation_data=check_stream, epochs=200,
               verbose=0, callbacks=[keras.callbacks.EarlyStopping(
                   monitor="val_loss", patience=25,
                   restore_best_weights=True)])

def r_squared(y_true, y_pred):
    resid = np.sum((y_true - y_pred) ** 2)
    total = np.sum((y_true - y_true.mean()) ** 2)
    return 1.0 - resid / total

test_scores = {}
print(f"{'modelo':<20}{'MSE prueba':>12}{'R2 prueba':>12}")
candidates = [("red densa tanh", main_model),
              ("capa SineFeatures", wave_model)]
for label, mdl in candidates:
    test_mse = mdl.evaluate(final_stream, verbose=0)
    test_mse = test_mse[0] if isinstance(test_mse, list) else test_mse
    pred = mdl.predict(x_final, verbose=0)
    test_scores[label] = (test_mse, r_squared(y_final, pred))
    print(f"{label:<20}{test_mse:>12.4f}{test_scores[label][1]:>12.4f}")
```

```
modelo                MSE prueba   R2 prueba
red densa tanh            0.0695      0.9494
capa SineFeatures         0.0578      0.9578
```

Ambos modelos alcanzan un R^2 cercano a 1 en datos que nunca vieron. Es la primera vez que se usa el conjunto de prueba, y por eso esta cifra es una estimación honesta.

12. Guardar y cargar modelos

Hay dos niveles de persistencia:

Qué se guarda	Método	Archivo	Para qué
Modelo completo: arquitectura, pesos y estado del optimizador	`model.save()`	`.keras`	Reanudar entrenamiento o desplegar
Solo los pesos	`model.save_weights()`	`.weights.h5`	Reutilizar pesos en una arquitectura idéntica

Las capas propias deben estar **registradas** (`register_keras_serializable`) y definir `get_config()`, para que Keras pueda reconstruirlas al cargar.

```
full_path = workdir / "modelo_ondas.keras"
wave_model.save(full_path)
restored = keras.models.load_model(full_path)

weights_path = workdir / "regresor.weights.h5"
main_model.save_weights(weights_path)
twin = dense_regressor("tanh")
```

```
twin.load_weights(weights_path)

probe = x_final[:5]
print("max |diferencia| modelo completo:",
      float(np.max(np.abs(wave_model.predict(probe, verbose=0)
                          - restored.predict(probe, verbose=0)))))
print("max |diferencia| solo pesos:     ",
      float(np.max(np.abs(main_model.predict(probe, verbose=0)
                          - twin.predict(probe, verbose=0)))))
```

```
max |diferencia| modelo completo: 0.0
max |diferencia| solo pesos:      0.0
```

Una diferencia de cero confirma que el modelo restaurado es idéntico al original.

13. Regularización

Regularizar es restringir el modelo para que prefiera soluciones simples y generalice mejor.

13.1. Penalizaciones L1, L2 y L1-L2

Se suma a la pérdida un término que depende de la magnitud de los pesos:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{datos}} + \lambda_1 \sum_j |w_j| + \lambda_2 \sum_j w_j^2.$$

- **L2** (*weight decay*) encoge todos los pesos de forma proporcional a su tamaño. Su gradiente es $2\lambda_2 w_j$.
- **L1** empuja pesos exactamente a cero, porque su subgradiente es la constante $\lambda_1 \operatorname{signo}(w_j)$. Eso produce modelos *dispersos*.
- **L1-L2** combina ambos efectos, igual que *elastic net* en modelos lineales.

Para ver el efecto, se entrena una red ancha sobre **solo 60 puntos**, una situación propensa al sobreajuste, con cada penalización.

```
small_x, small_y = x_fit[:60], y_fit[:60]
small_stream = to_stream(small_x, small_y, training=True)

def wide_net(reg=None, rate=0.0):
    layers = [keras.Input(shape=(1,))]
    for _ in range(2):
        layers.append(keras.layers.Dense(128, activation="tanh",
                                         kernel_regularizer=reg))
        if rate > 0:
            layers.append(keras.layers.Dropout(rate))
    layers.append(keras.layers.Dense(1))
    return keras.Sequential(layers)

configs = {
    "sin regularizar": dict(),
    "L2 (1e-3)": dict(reg=keras.regularizers.L2(1e-3)),
    "L1 (1e-4)": dict(reg=keras.regularizers.L1(1e-4)),
    "L1-L2": dict(reg=keras.regularizers.L1L2(l1=5e-5, l2=5e-4)),
```

```
    "Dropout (0.2)": dict(rate=0.2),
}
reg_scores = {}
print(f"{'variante':<16}{'MSE entren.':>12}{'MSE valid.':>12}{'|w| medio':>11}")
for label, kw in configs.items():
    keras.utils.set_random_seed(8)
    net = wide_net(**kw)
    net.compile(optimizer=keras.optimizers.Adam(0.01), loss="mse")
    net.fit(small_stream, epochs=400, verbose=0)
    fit_err = keras.ops.convert_to_numpy(
        keras.losses.mean_squared_error(small_y[:, 0], net(small_x)[:, 0]))
    val_err = keras.ops.convert_to_numpy(
        keras.losses.mean_squared_error(y_check[:, 0], net(x_check)[:, 0]))
    mean_w = np.mean([np.abs(w).mean() for w in net.get_weights()[::2]])
    reg_scores[label] = (float(fit_err), float(val_err))
    print(f"{label:<16}{float(fit_err):>12.4f}{float(val_err):>12.4f}"
          f"{mean_w:>11.4f}")
```

```
variante         MSE entren.  MSE valid.  |w| medio
sin regularizar       0.0567      0.1097     0.0984
L2 (1e-3)             0.0734      0.1148     0.0873
L1 (1e-4)             0.0645      0.1104     0.0847
L1-L2                 0.0695      0.1124     0.0861
Dropout (0.2)         0.1125      0.1559     0.0874
```

Cómo leer la tabla

Sin regularizar, el error de entrenamiento con 60 puntos queda claramente por debajo del de validación: el modelo aprendió parte del ruido. Las tres penalizaciones reducen el tamaño medio de los pesos y acortan esa brecha. En esta ejecución, sin embargo, ninguna mejora el error de validación, y *Dropout* con solo 60 puntos empeora ambos errores. Regularizar no garantiza un mejor resultado. λ y p son **hiperparámetros** que se ajustan con el conjunto de validación (ejercicio 5): si son demasiado grandes, el modelo subajusta.

13.2. *Dropout*

Durante el entrenamiento, *Dropout* apaga cada neurona con probabilidad p y multiplica las restantes por $1/(1-p)$, para que el valor esperado no cambie (*inverted dropout*). Cada lote ve así una subred distinta, y el modelo no puede depender de una única neurona. En **inferencia**, la capa no apaga nada. Keras gestiona ese cambio con el argumento `training`.

```
drop = keras.layers.Dropout(0.5, seed=1)
ones = tf.ones((1, 10))
print("entrenamiento:", drop(ones, training=True).numpy().round(1))
print("inferencia:   ", drop(ones, training=False).numpy().round(1))
```

```
entrenamiento: [[2. 0. 2. 0. 0. 0. 0. 2. 0. 2.]]
inferencia:    [[1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]]
```

En modo entrenamiento, aproximadamente la mitad de los valores son 0 y el resto vale 2. El promedio se mantiene cerca de 1. En inferencia, la capa es la identidad.

14. Normalización de activaciones

Las capas de normalización reescalan activaciones intermedias a media 0 y varianza 1, y después aplican una escala γ y un desplazamiento β aprendidos:

$$\hat{a} = \frac{a - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad \text{salida} = \gamma\,\hat{a} + \beta.$$

La diferencia entre las variantes está en **sobre qué ejes** se calculan μ y σ^2.

Variante	Estadísticas calculadas sobre	Depende del tamaño de lote	Uso típico
Batch Normalization	Los ejemplos del lote, por cada característica	Sí	Redes densas y convolucionales con lotes medianos o grandes
Layer Normalization	Las características de cada ejemplo	No	Redes recurrentes, *transformers*, lotes pequeños
Group Normalization	Grupos de características de cada ejemplo	No	Visión con lotes pequeños

Batch Normalization además guarda promedios móviles de μ y σ^2, que usa en inferencia. Por eso se comporta distinto en `training=True` y `training=False`. El experimento siguiente aplica las tres capas a un mismo tensor de 4 ejemplos con 6 características.

```
probe = tf.constant(np.random.default_rng(0).normal(
    loc=5.0, scale=3.0, size=(4, 6)).astype("float32"))
norms = {
    "BatchNorm": keras.layers.BatchNormalization(),
    "LayerNorm": keras.layers.LayerNormalization(),
    "GroupNorm (2 grupos)": keras.layers.GroupNormalization(groups=2),
}
for label, layer in norms.items():
    out = layer(probe, training=True).numpy()
    print(f"{label:<21} media por columna: {out.mean(axis=0).round(2)}")
    print(f"{'':<21} media por fila:    {out.mean(axis=1).round(2)}")
```

```
BatchNorm             media por columna: [-0.  0.  0. -0. -0.  0.]
                      media por fila:    [ 0.48 -0.08 -0.87  0.48]
LayerNorm             media por columna: [-0.1   0.59 -0.12  0.09 -0.85  0.39]
                      media por fila:    [-0.  0.  0.  0.]
GroupNorm (2 grupos)  media por columna: [-0.17  0.47 -0.31 -0.21 -0.67  0.88]
                      media por fila:    [-0.  0.  0.  0.]
```

BatchNorm deja en cero la media de cada **columna** (característica a través del lote). *LayerNorm* deja en cero la media de cada **fila** (cada ejemplo). *GroupNorm* normaliza por separado cada mitad de la fila.

```python
def normed_net(make_norm):
    layers = [keras.Input(shape=(1,))]
    for _ in range(2):
        layers += [keras.layers.Dense(64), make_norm(),
                   keras.layers.Activation("relu")]
    return keras.Sequential(layers + [keras.layers.Dense(1)])

factories = {
    "sin normalizar": lambda: keras.layers.Identity(),
    "BatchNorm": keras.layers.BatchNormalization,
    "LayerNorm": keras.layers.LayerNormalization,
    "GroupNorm": lambda: keras.layers.GroupNormalization(groups=8),
}
norm_scores = {}
for label, factory in factories.items():
    keras.utils.set_random_seed(13)
    net = normed_net(factory)
    net.compile(optimizer=keras.optimizers.Adam(0.01), loss="mse")
    h = net.fit(fit_stream, validation_data=check_stream, epochs=40, verbose=0)
    val_curve = h.history["val_loss"]
    norm_scores[label] = (val_curve[4], val_curve[-1])
    print(f"{label:<15} MSE validacion: epoca 5 = {val_curve[4]:.4f}"
          f" | epoca 40 = {val_curve[-1]:.4f}")
```

```
sin normalizar  MSE validacion: epoca 5 = 0.0865 | epoca 40 = 0.0776
BatchNorm       MSE validacion: epoca 5 = 0.3880 | epoca 40 = 0.1305
LayerNorm       MSE validacion: epoca 5 = 0.0897 | epoca 40 = 0.0966
GroupNorm       MSE validacion: epoca 5 = 0.2164 | epoca 40 = 0.0781
```

El resultado es instructivo porque la normalización **no** ayuda aquí:

- *BatchNorm* es la peor en validación. Durante el entrenamiento normaliza con las estadísticas de cada lote, pero en validación usa promedios móviles que se actualizan despacio (`momentum=0.99`). Con pocas actualizaciones, esos promedios todavía no representan bien las activaciones, y el modelo se evalúa con una normalización distinta de la que aprendió.
- *GroupNorm* empieza peor y alcanza a la red sin normalizar hacia la época 40.
- *LayerNorm* queda cerca de la red sin normalizar.

Las normalizaciones resuelven problemas de redes **profundas**, como activaciones que crecen o se desvanecen capa a capa. En una red de dos capas sobre una sola variable, ese problema no existe. Antes de añadir una técnica, conviene verificar que el problema que resuelve está presente.

15. Dimensiones de tensores y flujo de datos

Saber qué forma (*shape*) tiene cada tensor es la herramienta más útil para depurar una red. La letra B designa el tamaño del lote. En esta guía `BATCH = 64`, así que casi todos los lotes tienen $B = 64$, pero el conjunto de entrenamiento tiene 840 ejemplos ($840 = 13 \cdot 64 + 8$) y el último lote de cada época tiene $B = 8$. Los parámetros **no** dependen de B: el mismo peso se aplica a todos los ejemplos del lote.

Red densa principal (`dense_regressor("tanh")`, §8–§11):

Etapa	Shape	Significado de cada dimensión	Parámetros
Entrada `x`	$(B, 1)$	B ejemplos × 1 variable explicativa	—
`Dense(48, tanh)`	$(B, 48)$	48 activaciones ocultas por ejemplo	$1 \cdot 48 + 48 = 96$
`Dense(48, tanh)`	$(B, 48)$	48 activaciones ocultas por ejemplo	$48{\cdot}48{+}48 = 2\,352$
`Dense(1)`	$(B, 1)$	Una predicción $\hat{y}$ por ejemplo	$48 + 1 = 49$
Objetivo `y`	$(B, 1)$	Valor observado por ejemplo	—
Pérdida MSE	$()$, escalar	Promedio de los B errores al cuadrado	—

Red escrita a mano (§5.2): `x @ W_in` combina $(B, 1)$ con $(1, 32)$ y produce $(B, 32)$. El sesgo `b_in` tiene forma $(32,)$ y se **difunde** (*broadcasting*) a las B filas. Después `hidden @ W_out` combina $(B, 32)$ con $(32, 1)$ y produce $(B, 1)$. En total hay $32 + 32 + 32 + 1 = 97$ parámetros.

Modelo con `SineFeatures` (§7): la entrada $(B, 1)$ pasa por la capa de ondas, cuyos pesos son `omega` $(1, 16)$ y `phase` $(16,)$, y sale con forma $(B, 16)$. `Concatenate` la une con la entrada original, $(B, 1 + 16) = (B, 17)$, y `Dense(1)` produce $(B, 1)$ con $17 + 1 = 18$ parámetros.

Ejemplo de interpretación. En un lote completo, la primera capa densa multiplica $(64, 1) \times (1, 48)$ y suma el sesgo $(48,)$ a cada fila. El resultado son $64 \times 48 = 3\,072$ activaciones, calculadas con solo 96 parámetros. Si el lote fuera de 8 ejemplos, saldrían $8 \times 48 = 384$ activaciones con los mismos 96 parámetros. El código siguiente recorre la red con un lote real y confirma cada forma de la tabla.

```
xb, yb = next(iter(fit_stream))
sizes = [int(bx.shape[0]) for bx, _ in fit_stream]
print("lotes por epoca:", len(sizes), "| primero:", sizes[0],
      "| ultimo:", sizes[-1])
print(f"{'etapa':<22}{'shape':>10}{'parametros':>12}")
print(f"{'entrada x':<22}{str(tuple(xb.shape)):>10}{'-':>12}")
h = xb
for i, layer in enumerate(main_model.layers, start=1):
    h = layer(h)
    print(f"{'densa ' + str(i):<22}{str(tuple(h.shape)):>10}"
          f"{layer.count_params():>12}")
manual_hidden = tf.tanh(xb @ W_in + b_in)
print(f"{'manual: oculta':<22}{str(tuple(manual_hidden.shape)):>10}"
      f"{int(tf.size(W_in) + tf.size(b_in)):>12}")
print(f"{'manual: salida':<22}{str(tuple(tiny_net(xb).shape)):>10}"
      f"{int(tf.size(W_out) + tf.size(b_out)):>12}")
waves_out = wave_model.get_layer("ondas")(xb)
print(f"{'SineFeatures':<22}{str(tuple(waves_out.shape)):>10}"
      f"{wave_model.get_layer('ondas').count_params():>12}")
print(f"{'perdida (escalar)':<22}{str(tuple(mse(h, yb).shape)):>10}")
```

```
lotes por epoca: 14 | primero: 64 | ultimo: 8
etapa                       shape  parametros
entrada x                (64, 1)           -
densa 1                 (64, 48)          96
densa 2                 (64, 48)        2352
densa 3                  (64, 1)          49
manual: oculta          (64, 32)          64
manual: salida           (64, 1)          33
SineFeatures            (64, 16)          32
perdida (escalar)             ()
```

16. Hiperparámetros del modelo

Un **hiperparámetro** es una decisión que se fija antes de entrenar y que el descenso de gradiente no aprende. La tabla recoge los valores usados realmente en esta guía.

Hiperparámetro	Valor usado	Función	Al aumentarlo / disminuirlo
Partición	70 / 15 / 15 % de 1 200 puntos	Separar ajuste, selección y evaluación final	Más validación da decisiones más estables, pero deja menos datos para ajustar
Tamaño de lote	64	Ejemplos por paso de gradiente	Mayor: gradiente menos ruidoso y menos pasos por época. Menor: más ruido, que a veces ayuda a generalizar
Tasa de aprendizaje	0.05 (recta, SGD); 0.01 (red manual y principal, Adam); 0.005 (activaciones); 0.02 (`SineFeatures`)	Tamaño del paso de cada actualización	Muy alta: oscilación o divergencia. Muy baja: convergencia lenta
Épocas	30 (recta), 150 (red manual), 60 (activaciones), hasta 300 (principal), hasta 200 (ondas), 400 (regularización), 40 (normalización)	Pasadas completas por el entrenamiento	Más épocas reducen el subajuste, pero con pocos datos favorecen el sobreajuste
Anchura oculta	32 (manual), 48 (principal), 128 (regularización), 64 (normalización)	Capacidad del modelo	Mayor: más flexibilidad y más riesgo de sobreajuste. Menor: riesgo de subajuste
Activación	`tanh` (principal); comparación ReLU, tanh, GELU, SiLU	Introduce la no linealidad	Cambia la forma de las funciones representables y la velocidad de convergencia
Número de ondas	16 (`SineFeatures`)	Componentes periódicas disponibles	Más ondas: más flexibilidad para periodicidades complejas
`EarlyStopping`	`patience=20` (principal), 25 (ondas)	Épocas sin mejora antes de parar	Mayor: tolera mesetas, pero arriesga sobreajuste. Menor: puede parar antes de converger
Programador de tasa	`ReduceLROnPlateau`: `factor=0.5`, `patience=8`, `min_lr=1e-4`	Reduce la tasa cuando la validación se estanca	Un factor menor da recortes más bruscos
L2	$\lambda_2 = 10^{-3}$	Penaliza pesos grandes	Mayor: pesos más pequeños y posible subajuste
L1	$\lambda_1 = 10^{-4}$	Empuja pesos a cero	Mayor: modelo más disperso y posible subajuste

Hiperparámetro	Valor usado	Función	Al aumentarlo / disminuirlo
L1-L2	$\lambda_1 = 5 \cdot 10^{-5}$, $\lambda_2 = 5 \cdot 10^{-4}$	Combina ambos efectos	Igual que sus componentes
Dropout	$p = 0.2$	Apaga unidades al entrenar	Mayor: más regularización y más subajuste con pocos datos
Grupos de `GroupNorm`	8	Tamaño de los grupos normalizados	1 grupo equivale a *LayerNorm*; tantos grupos como canales normaliza cada canal por separado
Semillas	2026 (global) y semillas fijas por experimento	Reproducibilidad	No cambian el método, pero sí el resultado concreto de cada ejecución

En términos de **aprendizaje y generalización**, la tasa de aprendizaje y el tamaño de lote controlan la **convergencia**. La anchura, las épocas, la regularización y *Dropout* controlan el equilibrio entre **subajuste y sobreajuste**. *Patience* y los programadores de tasa automatizan decisiones que, sin ellos, habría que tomar mirando las curvas.

17. Errores frecuentes

1. **Evaluar con el conjunto de prueba durante el desarrollo.** El desempeño reportado deja de ser una estimación honesta. Usa validación para decidir y prueba solo al final.
2. **Usar `tf.constant` para parámetros.** No se pueden actualizar; deben ser `tf.Variable` o pesos de una capa.
3. **Olvidar que `GradientTape` solo registra lo que ocurre dentro del bloque `with`.** Si la predicción se calcula fuera, el gradiente es `None`.
4. **Activación en la capa de salida de una regresión.** Una ReLU final impide predecir negativos y una sigmoide limita la salida a $(0, 1)$.
5. **Confundir los dos tipos de archivo.** `save()` guarda el modelo completo (`.keras`) y `save_weights()` solo los pesos (`.weights.h5`). Cargar pesos exige una arquitectura idéntica.
6. **Capas propias sin `get_config`.** El modelo se guarda pero no se puede reconstruir al cargarlo.
7. **No fijar semillas al comparar variantes.** Las diferencias pueden deberse a la inicialización y no a la variante.

18. Ejercicios

1. **Gradiente a mano.** Repite el ejemplo de la §3 con $w = 2$, $b = 1$ y $\eta = 0.05$. Calcula la pérdida, los gradientes y los nuevos parámetros, y verifícalos con `GradientTape`.
2. **Tasa de aprendizaje.** En el entrenamiento manual de la red (§5), prueba $\eta \in \{0.001, 0.01, 0.1\}$ con Adam. ¿Qué pasa con la curva de pérdida en cada caso?
3. **Capacidad.** Cambia `HIDDEN` a 2, 8 y 128. Relaciona el número de parámetros con el error de validación.
4. **Capa propia.** Modifica `SineFeatures` para añadir una amplitud entrenable por onda. ¿Cambia el número de parámetros como esperabas?
5. **Regularización.** Barre $\lambda_2 \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$ en la red ancha y grafica el MSE de validación frente a λ_2 en escala logarítmica.

6. **Normalización y lote.** Repite la comparación de normalizaciones con `BATCH = 4`. ¿Qué variante se degrada más y por qué?

18.1. Respuestas orientativas

1. $\hat{y} = (3, 5, 7)$; errores $(0, 0, 1)$; $\mathcal{L} = 1/3 \approx 0.333$; $\partial_w \mathcal{L} = \frac{2}{3}(3) = 2$; $\partial_b \mathcal{L} = \frac{2}{3}(1) = 0.667$; $w = 1.9$, $b = 0.967$.
2. Con 0.001 la pérdida baja muy despacio. Con 0.01 el descenso es estable. Con 0.1 aparecen oscilaciones y la pérdida puede estancarse más alto.
3. Con 2 neuronas hay subajuste: la curva queda casi lineal. Con 128 el ajuste es bueno y, con tan pocos datos, puede empezar a sobreajustar. El número de parámetros es $3h + 1$.
4. Se añaden 16 parámetros (uno por onda) y el total pasa de 50 a 66.
5. La curva suele tener forma de U: errores altos en los extremos por sobreajuste o subajuste, y un mínimo intermedio.
6. *BatchNorm*, porque sus estadísticas se estiman con solo 4 ejemplos y son muy ruidosas. *LayerNorm* y *GroupNorm* no dependen del lote.

19. Glosario

Término	Significado
Época	Una pasada completa por el conjunto de entrenamiento
Lote (*batch*)	Subconjunto de ejemplos con el que se calcula un paso de gradiente
Gradiente	Vector de derivadas parciales de la pérdida respecto a los parámetros
Tasa de aprendizaje	Tamaño del paso de cada actualización
Subajuste / sobreajuste	Modelo demasiado simple o demasiado ajustado al ruido
`tf.Variable`	Tensor mutable que TensorFlow puede optimizar
`GradientTape`	Registro de operaciones para la diferenciación automática
Callback	Función que Keras ejecuta en momentos definidos del entrenamiento
Regularización	Restricción que favorece modelos simples
Normalización	Reescalado de activaciones a media 0 y varianza 1, con escala y desplazamiento aprendidos

20. Conclusiones académicas

Qué se aprendió. Entrenar una red es repetir un ciclo de cuatro pasos: predecir, medir la pérdida, calcular gradientes y actualizar parámetros. La guía lo construyó primero a mano (`tf.Variable` y `GradientTape`) y después con `tf.keras`. Así se ve que `compile()` y `fit()` automatizan exactamente ese ciclo, más la gestión de lotes, métricas y *callbacks*.

Qué demostraron los experimentos.

- La recta no puede representar la oscilación: su MSE de validación fue 1.099, muy por encima de la varianza del ruido (0.0625). Con una capa oculta de 32 neuronas `tanh`, la red manual

bajó a 0.094 en entrenamiento y 0.104 en validación. La no linealidad es lo que permite seguir la curva.
- En la comparación de activaciones, con la misma semilla y 60 épocas, gelu obtuvo el menor error de validación (0.0743) y tanh el mayor (0.1176).
- Con *callbacks*, el entrenamiento principal se detuvo en la época 105 de 300 posibles, conservando los mejores pesos.
- En prueba, la red densa alcanzó $R^2 = 0.949$ y el modelo con `SineFeatures` 0.958, con 50 parámetros frente a 2 497. Una capa diseñada para la estructura del problema (aquí, la periodicidad) puede igualar o superar a una red genérica mucho mayor.
- Con 60 puntos, la red ancha sin regularizar quedó con MSE 0.057 en entrenamiento y 0.110 en validación. Las penalizaciones redujeron los pesos y la brecha, pero no mejoraron la validación.
- La normalización no ayudó en esta red poco profunda: con *BatchNorm*, el MSE de validación en la época 40 fue 0.131, frente a 0.078 sin normalizar.

Cómo interpretar los resultados. Cada cifra describe **una ejecución** con una semilla, un conjunto sintético y un presupuesto de épocas concretos. Lo que se puede afirmar es el método: comparar variantes en igualdad de condiciones y decidir con validación. No se puede afirmar que una activación o una técnica sea mejor en general. Los resultados negativos (regularización y normalización sin mejora) son tan informativos como los positivos: muestran que una técnica sirve cuando el problema que resuelve está presente.

Limitaciones.

- Los datos son sintéticos, unidimensionales y con ruido gaussiano conocido.
- La mayoría de los experimentos usan una sola semilla, así que no se estimó la variabilidad entre ejecuciones.
- Los modelos son pequeños y se entrenaron en CPU.
- Los hiperparámetros no se buscaron de forma sistemática.

Qué conviene profundizar.

- Repetir las comparaciones con varias semillas e intervalos de confianza.
- Hacer una búsqueda de hiperparámetros (aleatoria o bayesiana) con validación cruzada.
- Usar datos reales con varias variables.
- Estudiar redes más profundas, donde la normalización y las conexiones residuales sí son decisivas.

Relación con el *Deep Learning*. Todo lo que se vio aquí se mantiene en los modelos grandes: tensores, diferenciación automática, optimizadores adaptativos, *callbacks*, regularización, normalización y guardado de modelos. Una red convolucional o un *transformer* cambian la arquitectura, pero se entrenan con el mismo ciclo y se depuran con las mismas preguntas: qué forma tiene cada tensor, qué está midiendo la pérdida y qué dice la validación.

21. Referencias

- Hortúa, H. J. *Deep Learning I — Clase 02: `01_Regresion.ipynb`*. Universidad El Bosque. Referencia conceptual de esta guía (enlaces en el recuadro de procedencia).
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*. MIT Press. Caps. 6–8. https://www.deeplearningbook.org/
- Ioffe, S. y Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. ICML.
- Ba, J. L., Kiros, J. R. y Hinton, G. E. (2016). *Layer Normalization*. arXiv:1607.06450.
- Wu, Y. y He, K. (2018). *Group Normalization*. ECCV.

- Srivastava, N. et al. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. JMLR 15.
- Documentación de TensorFlow y Keras: https://www.tensorflow.org/guide · https://keras.io/api/